HANDS-ON EXERCISE · MODULE 3 · SECTION 3.B

Bean & Brew: Building the Database

A guided to-do list: Designing, Creating, and Modifying Tables in MySQL

Course	Data Analyst Training Academy, Week 2: SQL Data Fundamentals
Guide reference	Student Guide, Module 3 (Working with Tables), Section 3.B (Creating and Modifying Tables), topics 3.B.1–3.B.15 (pp. 85–100)
Estimated time	90 minutes
Level	Beginner
Tools	MySQL Server 8.x and MySQL Workbench
Prerequisites	Modules 1–2 and Section 3.A: primary and foreign keys, relationships, column definitions, data types and NULL values
You will submit	One `.sql` script stage, commit and push to **Practice Exercises repository**

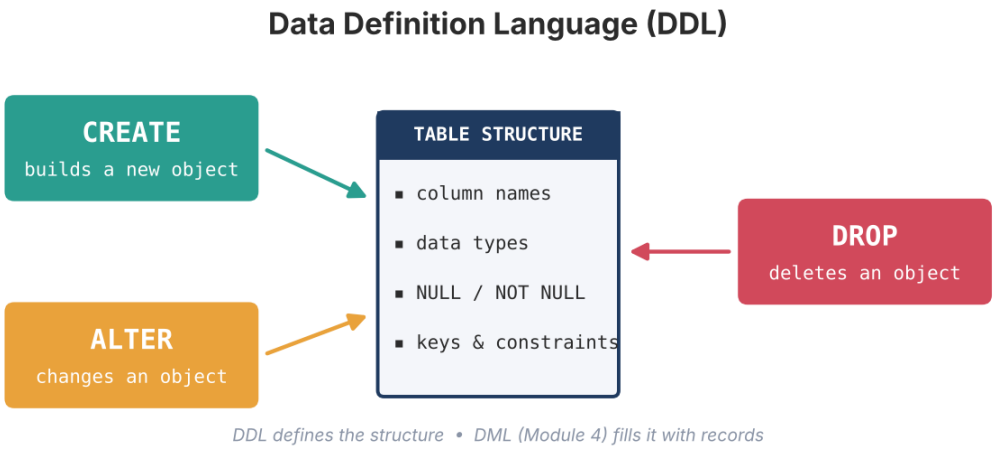


Figure 1. *The three DDL keywords. CREATE builds, ALTER changes and DROP deletes database objects (Student Guide 3.B.1).*

Learning objectives

By the end of this exercise you will be able to:

- Explain what Data Definition Language (DDL) is and name its three keywords: CREATE, ALTER and DROP (3.B.1).
- Write `CREATE TABLE` statements with primary keys, `AUTO_INCREMENT`, defaults and foreign keys, in the right order (3.B.2, 3.B.7, 3.B.8).

- Change an existing table with `ALTER TABLE`: add and modify columns, set default values and add foreign keys (3.B.3, 3.B.4).
- Add, name, disable, drop and replace `UNIQUE`, `CHECK` and foreign key constraints, and predict what `ON DELETE` / `ON UPDATE` rules do (3.B.5–3.B.13).
- Use a temporary table as a scratch pad and drop tables safely (3.B.14, 3.B.15).

1. The Scenario

You have just joined **Bean & Brew Coffee Co.**, a growing three-store café chain in the Boston area, as a junior data analyst. The company's menu, customers and sales are still kept in spreadsheets that nobody fully trusts.

Before the team can write a single report, it needs a real relational database. The Operations Manager has asked you to **design it, build it and make it protect itself** from bad data. You will build it in a practice database called `beanbrew_dev`.

> **MEMO · From: Dana Reyes, Operations Manager · To: New Data Analyst**
>
> Welcome to the team! Our menu, customers and sales still live in three messy spreadsheets. Before we can trust any report, I need a proper database. Please:
>
> - Follow the agreed design: proper keys, relationships and the right data type for every column.
> - Build the **beanbrew_dev** database: categories, products, customers, orders, and our staff list (who reports to whom).
> - Make the database protect itself: no duplicate customer emails, no $0.00 prices, valid zip codes, and **Accounting must never lose order history**.
> - Add our three stores and what each one stocks.
>
> One rule above all: **build in a practice database, and never DROP anything you have not double-checked.** Tables are easy to delete and hard to rebuild. — Dana

Dana's requests have been turned into a **to-do list** (Section 4). Each task gives you her **Request**, step-by-step **Instructions**, an **Explanation** of the concept, the **SQL to write**, and the **Expected result**, so you can check your work before ticking the box.

> **HOW TO USE THIS EXERCISE**
>
> - Work through the to-do list **in order**. Later tasks depend on earlier ones (table names, IDs and row counts change as you go).
> - **Type** each statement yourself rather than copying and pasting it. The only exception is the **starter-data block in Task 1.7**, which **you may paste**.
> - Some tasks are marked **Predict, then run**. Write your prediction down first; the error is part of the lesson.
> - Made a mess? Re-run the setup statements in Section 3 to start from an empty database, then begin again at Part 1.

2. Quick Concept Refresher

DDL (Data Definition Language) is the part of SQL you use to build the **structure** of a database: tables, columns, keys and rules. In Module 4 you will use DML to work with the **records** inside that structure. The table below summarises the DDL syntax from Section 3.B of the Student Guide.

Statement	Purpose	Syntax (MySQL 8)
CREATE TABLE 3.B.2	Build a new table	`CREATE TABLE t ( col TYPE [NOT NULL] [DEFAULT v], CONSTRAINT name ... );`
ALTER TABLE 3.B.3–3.B.4	Add / modify columns, set a default	`ALTER TABLE t ADD col TYPE; ALTER TABLE t MODIFY COLUMN col TYPE; ALTER TABLE t ALTER COLUMN col SET DEFAULT v;`
ADD CONSTRAINT 3.B.5–3.B.12	Add a named rule	`ALTER TABLE t ADD CONSTRAINT name UNIQUE (col) | CHECK (expr) | FOREIGN KEY ...;`
CREATE TEMPORARY TABLE 3.B.14	Build a scratch table for this session only	`CREATE TEMPORARY TABLE t ( ... );`
DROP 3.B.13, 3.B.15	Remove a rule or a table	`ALTER TABLE t DROP CONSTRAINT name; DROP TABLE t;`

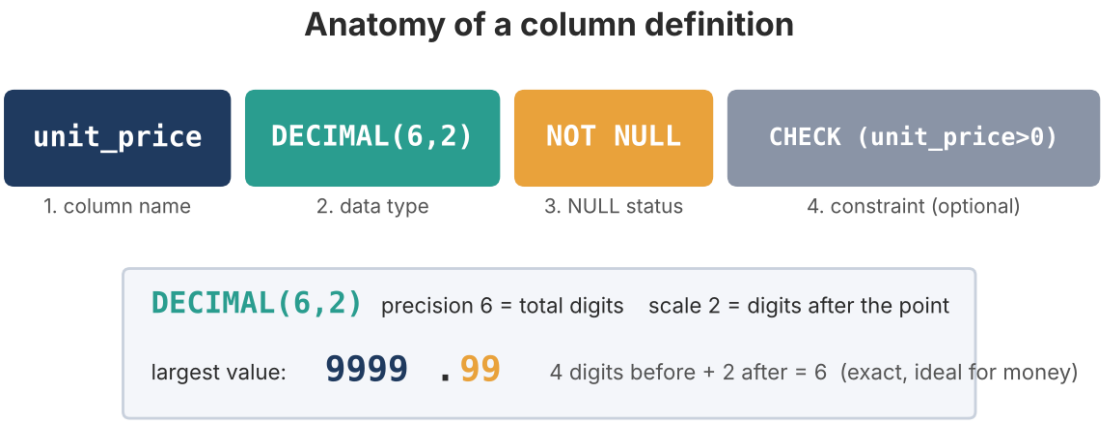


Figure 2. *Each column in a CREATE TABLE statement has a name, a data type, a NULL status and optional constraints (3.B.2, 3.B.5).*

Four rules to work by

1. **Plan before you build.** Check each table's keys, data types and NULL status against Figure 3 and the Table reference before you type. Changing a table full of data is much harder than changing a plan.
2. **Parents before children.** A foreign key can only point to a table that already exists, so create `category` before `product`, and `customer` and `product` before `orders`. Drop in the reverse order.

3. **Name your constraints.** `fk_orders_product` and `chk_price_positive` tell you exactly what failed. Generated names such as `store_chk_1` have to be looked up with `SHOW CREATE TABLE`.
4. **Check your work with DESCRIBE.** After every `CREATE` or `ALTER`, run `DESCRIBE table;` and confirm each column's type, NULL status, key and default.

3. Setup: Create the beanbrew_dev Database

You will build every table yourself, so the setup only creates an empty practice database. Running these statements again at any time **resets** everything: all tables in `beanbrew_dev` are dropped.

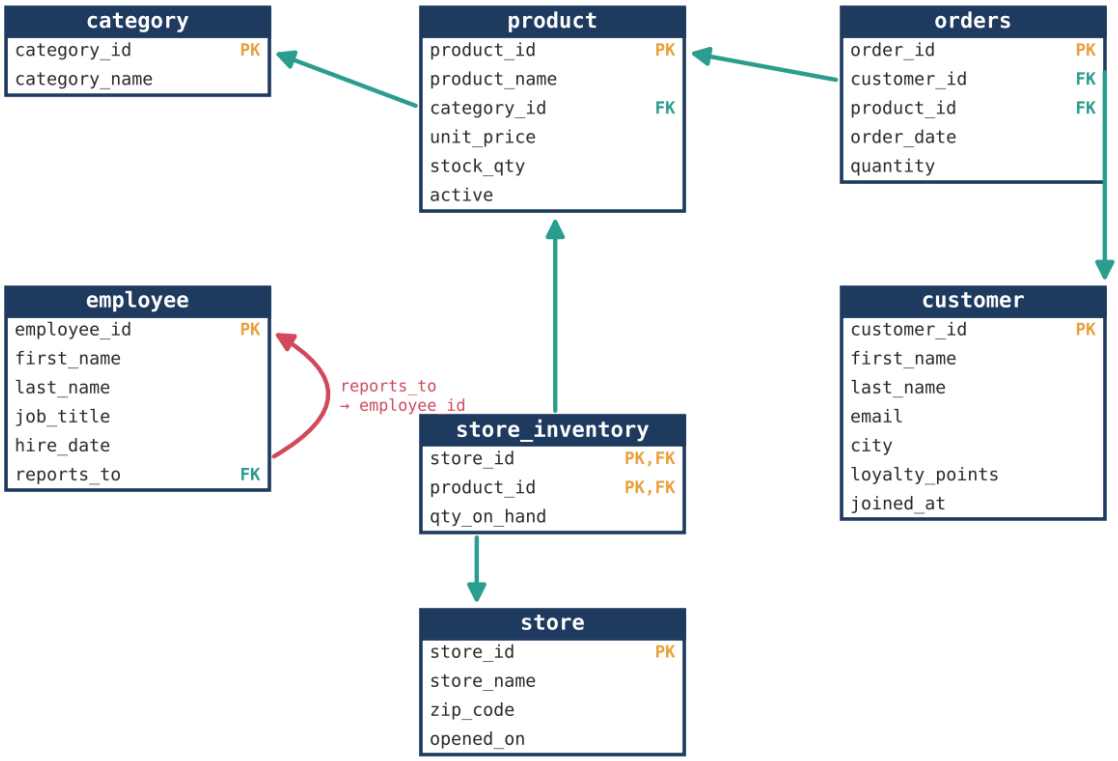
1. Open **MySQL Workbench** and connect to your local MySQL instance.
2. Open a new query tab (**File › New Query Tab**, or **Ctrl + T**).
3. Type the four statements below and run them with the **lightning-bolt** icon (Execute all).
4. In the Navigator panel, click the **refresh** icon next to SCHEMAS. You should now see `beanbrew_dev` with no tables yet.
5. `USE beanbrew_dev;` makes it the default database for this tab. Run it again whenever you open a new tab or reconnect. `SELECT DATABASE();` should return beanbrew_dev.

SQL TO WRITE

```
-- Setup: create (or reset) the practice database
DROP DATABASE IF EXISTS beanbrew_dev;
CREATE DATABASE beanbrew_dev;
USE beanbrew_dev;
SELECT DATABASE();
```

EXPECTED RESULT 1 row

DATABASE()
beanbrew_dev



PK = primary key FK = foreign key (arrow points to the parent table it references)

Red loop = recursive relationship: a foreign key that references its own table.

Figure 3. *The beanbrew_dev database you will build. Arrows run from each foreign key to the table it references.*

Table reference

This is the target design. You will build it step by step in Parts 1–3.

Table	Key columns and rules you will build
category	category_id PK (TINYINT UNSIGNED, AUTO_INCREMENT) · category_name NOT NULL
product	product_id PK (AUTO_INCREMENT) · product_name NOT NULL · category_id FK → category · unit_price DECIMAL(6,2) NOT NULL · stock_qty (nullable) · active NOT NULL DEFAULT TRUE
customer	customer_id PK (AUTO_INCREMENT) · first_name, last_name NOT NULL · email, city (nullable) · loyalty_points NOT NULL DEFAULT 0 · joined_at TIMESTAMP
orders	order_id PK · customer_id FK → customer · product_id FK → product · order_date NOT NULL · quantity DEFAULT 1
employee	employee_id PK (starts at 100) · reports_to FK → employee (recursive, nullable)
store / store_inventory	store_id PK · zip_code CHAR(5) with CHECK · inventory PK is composite (store_id, product_id)

TROUBLESHOOTING

- **Error 1046: No database selected.** Run `USE beanbrew_dev;` or double-click beanbrew_dev in the Navigator.
- **Error 1050: Table already exists.** You ran a `CREATE TABLE` twice. Check with `SHOW TABLES;` and carry on; or reset (Section 3).
- **Error 3780: Referencing column and referenced column are incompatible.** The foreign key column and the primary key it points to must have exactly the same data type (for example both `TINYINT UNSIGNED`).
- **Typed quotes look curly (’ ”)?** Re-type them in Workbench. SQL needs straight quotes.

4. Your To-Do List

This is your checklist. Work through the items from top to bottom and tick each ☐ box once your result matches the expected output. Every task in Parts 1–6 shows the same checkbox, so you can tick it there or here.

Done	Task	To do	Guide ref.
☐	**0**	Create the practice database	Section 3
☐	**1.1**	Create the category table	3.B.2
☐	**1.2**	Create the product table with a foreign key	3.B.2
☐	**1.3**	Predict, then run: a child table before its parent	3.B.7
☐	**1.4**	Create the customer table	3.B.2
☐	**1.5**	Create the orders table (now it works)	3.B.2
☐	**1.6**	Create the employee table (recursive relationship)	3.B.7
☐	**1.7**	Load the starter data (provided)	3.B.8
☐	**2.1**	Add a column	3.B.3
☐	**2.2**	Modify a column and add a phone number	3.B.3
☐	**2.3**	Predict, then run: set a default value	3.B.4
☐	**3.1**	No duplicate emails (UNIQUE)	3.B.5
☐	**3.2**	No zero prices (CHECK)	3.B.11
☐	**3.3**	Create the store table (column-level constraints)	3.B.6
☐	**3.4**	Track stock per store (composite primary key)	3.B.7
☐	**3.5**	Predict, then run: ON DELETE and ON UPDATE rules	3.B.9
☐	**3.6**	Disable, drop and replace a constraint	3.B.11–3.B.13

Done	Task	To do	Guide ref.
☐	**4.1**	Test a rule on a scratch table (temporary table)	3.B.14
☐	**4.2**	Drop a table (safely)	3.B.15
☐	**5.1**	Link staff to a home store (add a foreign key with ALTER TABLE) (optional)	3.B.3
☐	**5.2**	A sale price below the regular price (two-column CHECK) (optional)	3.B.3
☐	**R**	Answer the six reflection questions	Section 10
☐	**S**	Submit your script, screenshots and answers	Section 11

5. Part 1: Creating Tables with CREATE TABLE

Student Guide reference: 3.B.1, 3.B.2, 3.B.7, 3.B.8

Turn the design in Figure 3 into DDL. Build the parent tables first, then the tables that point to them. After each `CREATE TABLE`, use `DESCRIBE` to check what MySQL actually built.

☐ Task 1.1 · Create the category table 3.B.2, 3.B.8

Request: *"Start with the categories: Coffee, Tea, Bakery and Merchandise."*

INSTRUCTIONS

1. Make sure `beanbrew_dev` is the active database (Section 3, step 5).
2. Type the `CREATE TABLE` statement below. Note the parentheses, the commas between columns and the final semicolon.
3. Run it with **Ctrl + Enter**. The Output panel should show a green tick.
4. Run `DESCRIBE category;` and compare with the expected result.

EXPLANATION

Key idea: `CREATE TABLE name ( column TYPE [constraints], … );` A new table needs a name plus a name and definition for every column.

Why it works: `PRIMARY KEY` makes `category_id` required and unique. `AUTO_INCREMENT` makes it a surrogate key that MySQL fills in for you (1, 2, 3 …). `NOT NULL` makes the category name required.

SQL TO WRITE

```
-- Task 1.1: Create the category table
CREATE TABLE category
( category_id   TINYINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  category_name VARCHAR(30) NOT NULL
);

DESCRIBE category;
```

EXPECTED RESULT 2 rows

Field	Type	Null	Key	Default	Extra
category_id	tinyint unsigned	NO	PRI	NULL	auto_increment
category_name	varchar(30)	NO		NULL	

☐ **Check:** Two columns; category_id shows PRI and auto_increment.

☐ Task 1.2 · Create the product table with a foreign key 3.B.2, 3.B.4, 3.B.7

Request: *"Next, the menu. Every product belongs to a category, and new products should be active by default."*

INSTRUCTIONS

1. Type the statement below. The `CONSTRAINT … FOREIGN KEY … REFERENCES` clause goes after the last column, separated by a comma.
2. Check that `category_id` here has **exactly** the same type as `category.category_id`: `TINYINT UNSIGNED`.
3. Run it, then run `DESCRIBE product;`.

EXPLANATION

Key idea: `CONSTRAINT name FOREIGN KEY (column) REFERENCES parent(column) ON DELETE … ON UPDATE …`

Why it works: The foreign key means a product can only use a `category_id` that already exists in `category`. `ON DELETE RESTRICT` stops anyone deleting a category that still has products. `ON UPDATE CASCADE` copies a changed category number to its products. `BOOLEAN` is stored as `tinyint(1)` and `DEFAULT TRUE` shows as `1`. `MUL` in the Key column marks an indexed foreign key.

SQL TO WRITE

```
-- Task 1.2: Create the product table with a foreign key
CREATE TABLE product
( product_id   INT PRIMARY KEY AUTO_INCREMENT,
  product_name VARCHAR(60) NOT NULL,
  category_id  TINYINT UNSIGNED NOT NULL,
  unit_price   DECIMAL(6,2) NOT NULL,
  stock_qty    INT UNSIGNED,
  active       BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT fk_product_category FOREIGN KEY (category_id)
    REFERENCES category(category_id)
    ON DELETE RESTRICT ON UPDATE CASCADE
);

DESCRIBE product;
```

EXPECTED RESULT 6 rows

Field	Type	Null	Key	Default	Extra
product_id	int	NO	PRI	NULL	auto_increment
product_name	varchar(60)	NO		NULL	
category_id	tinyint unsigned	NO	MUL	NULL	

Field	Type	Null	Key	Default	Extra
unit_price	decimal(6,2)	NO		NULL	
stock_qty	int unsigned	YES		NULL	
active	tinyint(1)	NO		1	

☐ **Check:** Six columns; stock_qty is the only nullable column; active defaults to 1.

☐ Task 1.3 · Predict, then run: a child table before its parent 3.B.7

Request: *"A coworker tried to create the orders table first, before customers existed. What happens?"*

INSTRUCTIONS

1. Read the statement and write down your prediction before you run it.
2. Run it exactly as written.
3. Copy the error code and message from the Output panel and take a screenshot for your submission.
4. Run `SHOW TABLES;` and confirm that `orders` was **not** created.

EXPLANATION

Key idea: A foreign key can only reference a table that already exists.

Why it works: `orders` references `customer(customer_id)`, but there is no `customer` table yet, so MySQL refuses to create `orders` at all (Error 1824). Build order matters: parents first, children second.

SQL TO WRITE

```
-- Task 1.3: Predict, then run: a child table before its parent
CREATE TABLE orders
( order_id    INT PRIMARY KEY AUTO_INCREMENT,
  customer_id INT NOT NULL,
  product_id  INT NOT NULL,
  order_date  DATE NOT NULL,
  quantity    SMALLINT UNSIGNED NOT NULL DEFAULT 1,
  CONSTRAINT fk_orders_customer FOREIGN KEY (customer_id)
    REFERENCES customer(customer_id)
    ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_orders_product FOREIGN KEY (product_id)
    REFERENCES product(product_id)
    ON DELETE RESTRICT ON UPDATE CASCADE
);
```

WHAT YOU WILL SEE

```
Error Code: 1824.   Failed to open the referenced table 'customer'
```

☐ **Check:** The statement fails; SHOW TABLES still lists only category and product.

☐ Task 1.4 · Create the customer table 3.B.2, 3.B.4

Request: *"Now the customer list. Loyalty points start at zero, and we want to know when each person joined."*

INSTRUCTIONS

1. Type the statement below. Notice which columns have `NOT NULL` and which do not.
2. Run it, then run `DESCRIBE customer;`.
3. Find the two columns with a default value. What are the defaults?

EXPLANATION

Key idea: `DEFAULT value` supplies a value when none is given. `CURRENT_TIMESTAMP` records the moment the row is added.

Why it works: email and city are optional, so they allow NULL. `loyalty_points` is `NOT NULL DEFAULT 0`, so a new customer gets 0 points instead of NULL. `joined_at` is a `TIMESTAMP` that fills itself in automatically.

SQL TO WRITE

```
-- Task 1.4: Create the customer table
CREATE TABLE customer
( customer_id    INT PRIMARY KEY AUTO_INCREMENT,
  first_name     VARCHAR(30) NOT NULL,
  last_name      VARCHAR(30) NOT NULL,
  email          VARCHAR(100),
  city           VARCHAR(40),
  loyalty_points INT UNSIGNED NOT NULL DEFAULT 0,
  joined_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DESCRIBE customer;
```

EXPECTED RESULT 7 rows

Field	Type	Null	Key	Default	Extra
customer_id	int	NO	PRI	NULL	auto_increment
first_name	varchar(30)	NO		NULL	
last_name	varchar(30)	NO		NULL	
email	varchar(100)	YES		NULL	
city	varchar(40)	YES		NULL	
loyalty_points	int unsigned	NO		0	
joined_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

☐ **Check:** Seven columns; loyalty_points defaults to 0; joined_at defaults to CURRENT_TIMESTAMP.

☐ Task 1.5 · Create the orders table (now it works) 3.B.2, 3.B.7

Request: *"Customers and products exist now. Try the orders table again."*

INSTRUCTIONS

1. Re-run the exact `CREATE TABLE orders` statement from Task 1.3. You do not need to change anything.
2. Run `SHOW TABLES;` and confirm four tables.

> **EXPLANATION**
>
> **Key idea:** Same statement, different result: both parent tables now exist.
>
> **Why it works:** `orders` is the linking table that resolves the customer–product many-to-many relationship. Each of its two foreign keys points to a parent that is now in place. `ON DELETE RESTRICT` on both protects order history: a parent row cannot be deleted while an order still points to it.

SQL TO WRITE

```
-- Task 1.5: Create the orders table (now it works)
-- (re-run the CREATE TABLE orders statement from Task 1.3)

SHOW TABLES;
```

EXPECTED RESULT 4 rows

Tables_in_beanbrew_dev
category
customer
orders
product

☐ **Check:** SHOW TABLES returns 4 rows.

☐ Task 1.6 · Create the employee table (recursive relationship) 3.B.7, 3.B.8

Request: *"I also need the staff list, including who reports to whom. Our employee numbers start at 100."*

INSTRUCTIONS

1. Type the statement below. The foreign key `reports_to` points to `employee_id` in the **same** table.
2. Run it.
3. Run the `ALTER TABLE … AUTO_INCREMENT = 100` statement so that the first employee gets ID 100.
4. Run `SHOW TABLES;` and confirm five tables.

EXPLANATION

Key idea: A recursive relationship is a foreign key that references the primary key of its own table. Its column must have a different name.

Why it works: Every manager is also an employee, so there is no need for a separate manager table. `reports_to` is nullable because the top person (Dana) has no manager. `ON DELETE SET NULL` means that if a manager leaves, their staff's `reports_to` becomes NULL instead of blocking the delete. `AUTO_INCREMENT = 100` sets the starting value.

SQL TO WRITE

```
-- Task 1.6: Create the employee table (recursive relationship)
CREATE TABLE employee
( employee_id INT PRIMARY KEY AUTO_INCREMENT,
  first_name  VARCHAR(30) NOT NULL,
  last_name   VARCHAR(30) NOT NULL,
  job_title   VARCHAR(40) NOT NULL,
  hire_date   DATE NOT NULL,
  reports_to  INT,
  CONSTRAINT fk_employee_manager FOREIGN KEY (reports_to)
    REFERENCES employee(employee_id)
    ON DELETE SET NULL
);

ALTER TABLE employee AUTO_INCREMENT = 100;

SHOW TABLES;
```

EXPECTED RESULT 5 rows

Tables_in_beanbrew_dev
category
customer
employee
orders
product

☐ **Check:** Five tables. The employee table has a foreign key that points to itself.

☐ Task 1.7 · Load the starter data (provided) 3.B.8

Request: *"Here's what's in our spreadsheets today. Load it so we can test your rules."*

INSTRUCTIONS

1. This is the **one block you may paste**. `INSERT` is covered properly in Module 4. Here it simply gives your tables some rows.
2. Run the whole block with the lightning-bolt icon (Execute all).
3. Look at the result of `SELECT LAST_INSERT_ID();`. It shows the ID that `AUTO_INCREMENT` gave Dana.
4. Compare the row counts and the employee list with the expected results.

EXPLANATION

Key idea: `LAST_INSERT_ID()` returns the most recent value generated by `AUTO_INCREMENT` (3.B.8).

Why it works: No `product_id`, `customer_id` or `order_id` values are typed: MySQL generates them. Dates use `'YYYY-MM-DD'` in single quotes. Marcus, Sofia and Ken are only accepted because the `reports_to` values 100 and 101 already exist in the same table.

SQL TO WRITE

```
-- Task 1.7: Load the starter data (provided)
INSERT INTO category (category_name) VALUES
  ('Coffee'), ('Tea'), ('Bakery'), ('Merchandise');

INSERT INTO product (product_name, category_id, unit_price, stock_qty) VALUES
  ('House Blend Drip', 1, 2.75, 120),   ('Espresso Single', 1, 3.25, 80),
  ('Caramel Latte', 1, 4.95, 60),       ('Cold Brew', 1, 4.50, 45),
  ('Earl Grey', 2, 2.95, 50),           ('Chai Latte', 2, 4.75, 40),
  ('Blueberry Muffin', 3, 3.50, 24),    ('Butter Croissant', 3, 3.25, 30),
  ('Cinnamon Roll', 3, 3.95, 18),       ('Logo Travel Mug', 4, 14.99, 25),
  ('Pumpkin Spice Latte', 1, 5.45, 35), ('Ceramic Pour-Over Set', 4, 29.99, NULL);

INSERT INTO customer (first_name, last_name, email, city, loyalty_points) VALUES
  ('Maria', 'Lopez', 'maria.lopez@example.com', 'Boston', 340),
  ('James', 'Carter', 'james.carter@example.com', 'Cambridge', 120),
  ('Aisha', 'Khan', 'aisha.khan@example.com', 'Boston', 560),
  ('Tom', 'Nguyen', NULL, 'Somerville', 75),
  ('Priya', 'Shah', 'priya.shah@example.com', 'Boston', 0),
  ('Daniel', 'Brooks', 'daniel.brooks@example.com', 'Quincy', 210);

INSERT INTO orders (customer_id, product_id, order_date, quantity) VALUES
  (1, 3, '2026-09-01', 1), (2, 1, '2026-09-02', 2), (3, 7, '2026-09-03', 1),
  (1, 5, '2026-09-03', 1), (3, 11, '2026-09-04', 2), (4, 2, '2026-09-05', 1),
  (6, 10, '2026-09-06', 1), (1, 11, '2026-09-08', 1), (2, 8, '2026-09-09', 3);

INSERT INTO employee (first_name, last_name, job_title, hire_date, reports_to) VALUES
  ('Dana', 'Reyes', 'Operations Manager', '2021-03-15', NULL);
SELECT LAST_INSERT_ID();

INSERT INTO employee (first_name, last_name, job_title, hire_date, reports_to) VALUES
  ('Marcus', 'Hill', 'Shift Lead', '2023-06-01', 100),
  ('Sofia', 'Alvarez', 'Barista', '2024-09-10', 101),
  ('Ken', 'Ito', 'Barista', '2025-01-20', 101);

-- Check the row counts
SELECT 'category' AS table_name, COUNT(*) AS row_count FROM category
UNION ALL SELECT 'product', COUNT(*) FROM product
UNION ALL SELECT 'customer', COUNT(*) FROM customer
UNION ALL SELECT 'orders', COUNT(*) FROM orders
UNION ALL SELECT 'employee', COUNT(*) FROM employee;

SELECT employee_id, first_name, job_title, reports_to FROM employee;
```

EXPECTED RESULT LAST_INSERT_ID()

LAST_INSERT_ID()
100

EXPECTED RESULT row counts, 5 rows

table_name	row_count
category	4
product	12
customer	6
orders	9
employee	4

EXPECTED RESULT employee, 4 rows

employee_id	first_name	job_title	reports_to
100	Dana	Operations Manager	NULL
101	Marcus	Shift Lead	100
102	Sofia	Barista	101
103	Ken	Barista	101

☐ **Check:** LAST_INSERT_ID() returns 100; counts are 4, 12, 6, 9 and 4.

6. Part 2: Modifying Tables with ALTER TABLE

Student Guide reference: 3.B.3–3.B.4

Requirements change after a table is built. `ALTER TABLE` lets you change the structure without losing the rows already in it. Every statement starts with the table name, followed by `ADD`, `MODIFY COLUMN`, `ALTER COLUMN` or `DROP`.

☐ Task 2.1 · Add a column 3.B.3

Request: *"The website team wants a longer description for each product."*

INSTRUCTIONS

1. Write `ALTER TABLE product` and, on the next line, `ADD product_description TEXT;`.
2. Run it, then run `DESCRIBE product;`.
3. Confirm the new column appears at the end and allows NULL.

EXPLANATION

Key idea: `ALTER TABLE table ADD column TYPE;`

Why it works: A new column is added to every existing row. Because no value is supplied, the 12 existing products get NULL. `TEXT` holds long content without declaring a length.

SQL TO WRITE

```
-- Task 2.1: Add a column
ALTER TABLE product
ADD product_description TEXT;

DESCRIBE product;
```

EXPECTED RESULT 7 rows (last row shown)

Field	Type	Null	Key	Default	Extra
…	…	…	…	…	…
product_description	text	YES		NULL	

☐ **Check:** product now has 7 columns; product_description is text and nullable.

☐ Task 2.2 · Modify a column and add a phone number 3.B.3

Request: *"Some product names are being cut off. Make room for up to 100 characters. Also start collecting customer phone numbers."*

INSTRUCTIONS

1. Use `MODIFY COLUMN` to change `product_name` to `VARCHAR(100) NOT NULL`. Include `NOT NULL` again.
2. Run it. The Output panel reports **12 rows affected** because MySQL rebuilds every row.
3. Add a `phone VARCHAR(20)` column to `customer`.
4. Run `DESCRIBE customer;` and find the new column.

EXPLANATION

Key idea: `MODIFY COLUMN` replaces the column's **whole** definition: type, NULL status and default.

Why it works: If you wrote only `MODIFY COLUMN product_name VARCHAR(100);`, the column would silently become nullable, because anything you leave out is dropped. Always restate `NOT NULL` and any `DEFAULT`. Phone numbers are stored as text, not numbers: they can start with 0 or contain + and dashes, and nobody does arithmetic on them.

SQL TO WRITE

```
-- Task 2.2: Modify a column and add a phone number
ALTER TABLE product
MODIFY COLUMN product_name VARCHAR(100) NOT NULL;

ALTER TABLE customer
ADD phone VARCHAR(20);

DESCRIBE customer;
```

EXPECTED RESULT 8 rows (last row shown)

Field	Type	Null	Key	Default	Extra
…	…	…	…	…	…
phone	varchar(20)	YES		NULL	

☐ **Check:** MODIFY reports 12 rows affected; customer now has 8 columns.

☐ Task 2.3 · Predict, then run: set a default value 3.B.4

Request: *"Most new sign-ups are at our Boston stores. If nobody enters a city, assume Boston."*

INSTRUCTIONS

1. (a) Run the first statement, which uses the form shown on Student Guide page 89. Write down what happens.
2. (b) Run the second statement, which uses MySQL 8's `ALTER COLUMN … SET DEFAULT` form.
3. (c) Add a walk-in customer, Lena Park, giving only her first and last name.
4. (d) SELECT her row and check every column: which got a default, and which are NULL?

EXPLANATION

Key idea: In MySQL 8: `ALTER TABLE t ALTER COLUMN col SET DEFAULT value;` (or restate the full definition with `MODIFY COLUMN col TYPE DEFAULT value`).

Why it works: `MODIFY COLUMN` expects a full column definition, so `MODIFY COLUMN city SET DEFAULT …` is a syntax error (1064). As the guide notes, syntax varies by DBMS, so check the documentation. Lena's `city` became Boston and `loyalty_points` became 0 (both defaults). `email` has no default, so it is NULL. A default is used **only** when no value is supplied.

SQL TO WRITE

```
-- Task 2.3: Predict, then run: set a default value
-- (a) Guide form: fails in MySQL 8
ALTER TABLE customer MODIFY COLUMN city SET DEFAULT 'Boston';

-- (b) MySQL 8 form: works
ALTER TABLE customer
ALTER COLUMN city SET DEFAULT 'Boston';

-- (c) Add a walk-in customer
INSERT INTO customer (first_name, last_name) VALUES ('Lena', 'Park');

-- (d) Verify
SELECT customer_id, first_name, last_name, email, city, loyalty_points
FROM customer
WHERE last_name = 'Park';
```

EXPECTED ERROR (on purpose)

```
Error Code: 1064.   You have an error in your SQL syntax; check the manual that
corresponds to your MySQL server version for the right syntax to use near 'DEFAULT
'Boston'' at line 1
```

EXPECTED RESULT 1 row

customer_id	first_name	last_name	email	city	loyalty_points
7	Lena	Park	NULL	Boston	0

☐ **Check:** (a) Error 1064. (d) customer_id 7; city = Boston; loyalty_points = 0; email is NULL.

7. Part 3: Protecting the Data with Constraints

Student Guide reference: 3.B.5–3.B.13

Constraints are rules the database enforces on every insert and update, so bad data never gets in. In this part you will add named constraints, build two new tables with keys and checks, watch `ON DELETE` rules in action, and replace a constraint.

☐ Task 3.1 · No duplicate emails (UNIQUE) 3.B.5, 3.B.6, 3.B.10

Request: *"Marketing keeps getting two loyalty accounts for one person. Make every email unique."*

INSTRUCTIONS

1. Add a named `UNIQUE` constraint, `uq_customer_email`, on `customer(email)`.
2. Run it. It succeeds even though Tom and Lena both have a NULL email.
3. **Predict, then run:** try to add a second customer with Maria's email. Take a screenshot of the error.

EXPLANATION

Key idea: `ALTER TABLE t ADD CONSTRAINT name UNIQUE (column);` Columns with a UNIQUE constraint are sometimes called candidate keys.

Why it works: UNIQUE rejects a duplicate value (Error 1062) and the message names the constraint, which is why a meaningful name helps. NULL is a non-value, so it never equals another NULL: MySQL allows several NULLs in a UNIQUE column.

SQL TO WRITE

```
-- Task 3.1: No duplicate emails (UNIQUE)
ALTER TABLE customer
ADD CONSTRAINT uq_customer_email UNIQUE (email);

-- Predict, then run
INSERT INTO customer (first_name, last_name, email)
VALUES ('Maria', 'Lopez', 'maria.lopez@example.com');
```

EXPECTED ERROR (on purpose)

```
Error Code: 1062.   Duplicate entry 'maria.lopez@example.com' for key
'customer.uq_customer_email'
```

☐ **Check:** The constraint is added; the duplicate INSERT fails with Error 1062.

☐ Task 3.2 · No zero prices (CHECK) 3.B.11

Request: *"Last year someone set a price to $0.00 by accident. Make that impossible."*

INSTRUCTIONS

1. Add a named table-level `CHECK` constraint, `chk_price_positive`, requiring `unit_price > 0`.
2. Run it. The Output panel reports **12 rows affected** because every existing row is checked.
3. **Predict, then run:** try to add a product priced at 0.00.

EXPLANATION

Key idea: `ADD CONSTRAINT name CHECK (expression)`. The expression must evaluate to TRUE or FALSE.

Why it works: Each time a row is inserted or updated, MySQL tests the new value against the condition. If it is FALSE, the change is rejected (Error 3819). Existing rows are checked when the constraint is added, so it would fail if any current price were 0.

SQL TO WRITE

```
-- Task 3.2: No zero prices (CHECK)
ALTER TABLE product
ADD CONSTRAINT chk_price_positive CHECK (unit_price > 0);

-- Predict, then run
INSERT INTO product (product_name, category_id, unit_price)
VALUES ('Free Sample Cup', 1, 0.00);
```

EXPECTED ERROR (on purpose)

```
Error Code: 3819.   Check constraint 'chk_price_positive' is violated.
```

☐ **Check:** Adding the constraint affects 12 rows; the 0.00 INSERT fails with Error 3819.

☐ Task 3.3 · Create the store table (column-level constraints) 3.B.6, 3.B.10–3.B.12

Request: *"Add our three stores: Back Bay, Kendall Square and Seaport. Store names must be unique and zip codes must be five characters."*

INSTRUCTIONS

1. Type the `CREATE TABLE store` statement. This time `UNIQUE` and `CHECK` are written inside the column definitions, without names.
2. Run `SHOW CREATE TABLE store;` and find the names MySQL generated for the two constraints. Write them down.
3. Run the provided `INSERT` for the three stores.
4. **Predict, then run:** try a store with the 4-character zip code '0212'.

EXPLANATION

Key idea: `column TYPE CHECK (expression)` creates a column-level check. Unnamed constraints get generated names such as `store_chk_1`.

Why it works: Because you did not name them, MySQL called the check `store_chk_1` and the unique key `store_name` (after the column). You will need the name `store_chk_1` in Task 3.6. `CHAR(5)` already caps the length at 5, and the check adds the minimum, so the length must be exactly 5.

SQL TO WRITE

```
-- Task 3.3: Create the store table (column-level constraints)
CREATE TABLE store
```

```
( store_id   TINYINT UNSIGNED PRIMARY KEY AUTO_INCREMENT,
  store_name VARCHAR(40) NOT NULL UNIQUE,
  zip_code   CHAR(5) NOT NULL CHECK (CHAR_LENGTH(zip_code) = 5),
  opened_on  DATE
);

SHOW CREATE TABLE store;

INSERT INTO store (store_name, zip_code, opened_on) VALUES
  ('Back Bay', '02116', '2019-05-01'),
  ('Kendall Square', '02142', '2022-02-14'),
  ('Seaport', '02210', '2025-08-30');

-- Predict, then run
INSERT INTO store (store_name, zip_code) VALUES ('Dorchester', '0212');
```

WHAT YOU WILL SEE (SHOW CREATE TABLE, excerpt)

```
UNIQUE KEY `store_name` (`store_name`),
CONSTRAINT `store_chk_1` CHECK ((char_length(`zip_code`) = 5))
```

EXPECTED ERROR (on purpose)

```
Error Code: 3819.   Check constraint 'store_chk_1' is violated.
```

☐ **Check:** Three stores inserted; the generated names are store_chk_1 and store_name; '0212' is rejected.

☐ Task 3.4 · Track stock per store (composite primary key) 3.B.7, 3.B.9

Request: *"Each store stocks different amounts of each product. I need one row per store per product."*

INSTRUCTIONS

1. Type the `CREATE TABLE store_inventory` statement. The primary key is a table-level constraint over **two** columns.
2. Run it, then run the provided `INSERT` (5 rows).
3. **Predict, then run:** try to add store 1 / product 1 a second time.

EXPLANATION

Key idea: `CONSTRAINT name PRIMARY KEY (col1, col2)` creates a composite key: the combination must be unique.

Why it works: Store 1 can appear many times and product 1 can appear many times, but the pair (1, 1) only once (Error 1062 on '1-1'). This table also resolves the store–product many-to-many relationship. `ON DELETE CASCADE` means that if a store or product is deleted, its inventory rows go with it. Inventory has no meaning without its store.

SQL TO WRITE

```
-- Task 3.4: Track stock per store (composite primary key)
CREATE TABLE store_inventory
( store_id    TINYINT UNSIGNED,
```

```
  product_id  INT,
  qty_on_hand SMALLINT UNSIGNED NOT NULL DEFAULT 0,
  CONSTRAINT pk_store_inventory PRIMARY KEY (store_id, product_id),
  CONSTRAINT fk_inv_store FOREIGN KEY (store_id)
    REFERENCES store(store_id) ON DELETE CASCADE,
  CONSTRAINT fk_inv_product FOREIGN KEY (product_id)
    REFERENCES product(product_id) ON DELETE CASCADE
);

INSERT INTO store_inventory (store_id, product_id, qty_on_hand) VALUES
  (1, 1, 40), (1, 7, 12), (2, 1, 35), (3, 1, 20), (3, 7, 8);

-- Predict, then run
INSERT INTO store_inventory (store_id, product_id, qty_on_hand)
VALUES (1, 1, 99);
```

EXPECTED ERROR (on purpose)

```
Error Code: 1062.   Duplicate entry '1-1' for key 'store_inventory.PRIMARY'
```

☐ **Check:** 5 inventory rows inserted; the duplicate (1, 1) fails with Error 1062.

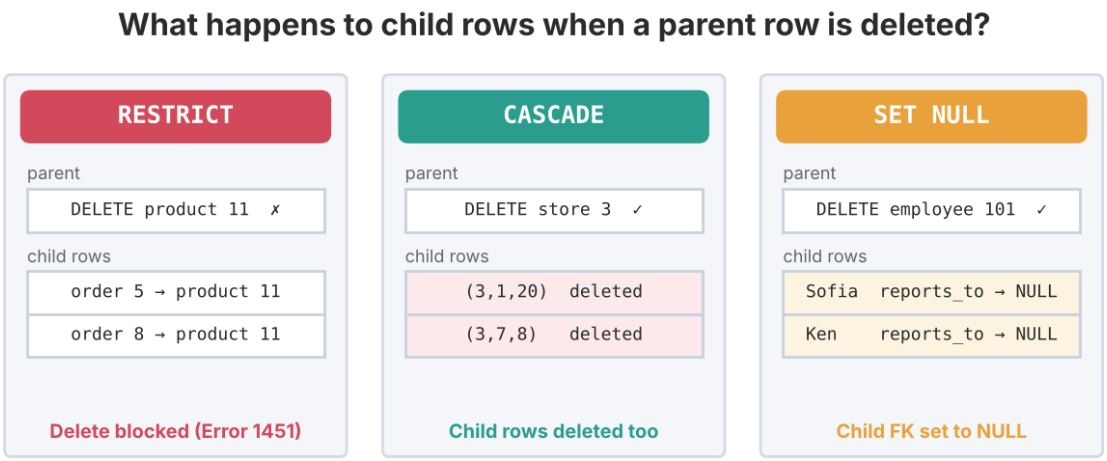


Figure 4. *RESTRICT blocks the delete, CASCADE removes the child rows, SET NULL empties the child's foreign key (3.B.9).*

☐ Task 3.5 · Predict, then run: ON DELETE and ON UPDATE rules 3.B.9

Request: *"Before we go live, show me exactly what happens when someone deletes or renumbers a parent row."*

INSTRUCTIONS

1. Before each step, look at Figure 4 and predict the outcome.
2. (a) Try to delete product 11 (Pumpkin Spice Latte). Two orders reference it. Take a screenshot.
3. (b) Delete store 3 (Seaport), then SELECT all inventory rows.
4. (c) Delete employee 101 (Marcus, the shift lead), then SELECT the employees.
5. (d) Renumber category 4 to 40, SELECT the products in category 40, then change it back to 4.

EXPLANATION

Key idea: `RESTRICT` blocks the change. `CASCADE` passes it on to the child rows. `SET NULL` empties the child's foreign key.

Why it works: (a) `fk_orders_product` is `ON DELETE RESTRICT`, so order history is protected (Error 1451), as Accounting asked. (b) `fk_inv_store` is `ON DELETE CASCADE`, so store 3's two inventory rows were deleted automatically. (c) `fk_employee_manager` is `ON DELETE SET NULL`, so Sofia and Ken now have no manager. (d) `fk_product_category` is `ON UPDATE CASCADE`, so the two Merchandise products followed the new number.

SQL TO WRITE

```
-- Task 3.5: Predict, then run: ON DELETE and ON UPDATE rules
-- (a) RESTRICT
DELETE FROM product WHERE product_id = 11;

-- (b) CASCADE
DELETE FROM store WHERE store_id = 3;
SELECT * FROM store_inventory;

-- (c) SET NULL
DELETE FROM employee WHERE employee_id = 101;
SELECT employee_id, first_name, job_title, reports_to FROM employee;

-- (d) ON UPDATE CASCADE
UPDATE category SET category_id = 40 WHERE category_id = 4;
SELECT product_id, product_name, category_id FROM product WHERE category_id = 40;
UPDATE category SET category_id = 4 WHERE category_id = 40;
```

EXPECTED ERROR (on purpose), step (a)

```
Error Code: 1451.   Cannot delete or update a parent row: a foreign key constraint fails
(`beanbrew_dev`.`orders`, CONSTRAINT `fk_orders_product` FOREIGN KEY (`product_id`)
REFERENCES `product` (`product_id`) ON DELETE RESTRICT ON UPDATE CASCADE)
```

EXPECTED RESULT step (b), 3 rows

store_id	product_id	qty_on_hand
1	1	40
1	7	12
2	1	35

EXPECTED RESULT step (c), 3 rows

employee_id	first_name	job_title	reports_to
100	Dana	Operations Manager	NULL
102	Sofia	Barista	NULL
103	Ken	Barista	NULL

EXPECTED RESULT step (d), 2 rows

product_id	product_name	category_id
10	Logo Travel Mug	40
12	Ceramic Pour-Over Set	40

☐ **Check:** (a) Error 1451. (b) 3 inventory rows left. (c) reports_to is NULL for Sofia and Ken. (d) 2 products followed the change; category 4 restored.

☐ Task 3.6 · Disable, drop and replace a constraint 3.B.11–3.B.13

Request: *"The zip-code rule only checks the length, so '0211A' would get through. Replace it with a rule that allows digits only, and give it a proper name."*

INSTRUCTIONS

1. (a) Turn the generated check `store_chk_1` off with `NOT ENFORCED`, then back on with `ENFORCED`.
2. (b) Drop `store_chk_1` with `DROP CHECK`.
3. (c) Add a new named constraint, `chk_zip_digits`, that accepts exactly five digits.
4. (d) **Predict, then run:** insert a store with zip code '0211A'.
5. (e) List the constraints on `store` from `information_schema` and confirm the result.

EXPLANATION

Key idea: `ALTER CHECK name [NOT] ENFORCED` turns a check off or on. `DROP CHECK name` (or `DROP CONSTRAINT name`) removes it. You need the constraint's name.

Why it works: `CHAR_LENGTH(zip_code) = 5` is TRUE for '0211A', so the old rule let letters through. `REGEXP '^[0-9]{5}$'` means "start, exactly five digits, end". Because the new constraint has a meaningful name, the error message tells you exactly which rule failed. When re-enabled with `ENFORCED`, the existing 2 rows are checked again.

SQL TO WRITE

```
-- Task 3.6: Disable, drop and replace a constraint
-- (a) Turn off, then on again
ALTER TABLE store
ALTER CHECK store_chk_1 NOT ENFORCED;
ALTER TABLE store
ALTER CHECK store_chk_1 ENFORCED;

-- (b) Drop the old rule
ALTER TABLE store
DROP CHECK store_chk_1;

-- (c) Add a better, named rule
ALTER TABLE store
ADD CONSTRAINT chk_zip_digits CHECK (zip_code REGEXP '^[0-9]{5}$');

-- (d) Predict, then run
INSERT INTO store (store_name, zip_code) VALUES ('Dorchester', '0211A');

-- (e) Verify
SELECT constraint_name, constraint_type
FROM information_schema.table_constraints
WHERE table_schema = 'beanbrew_dev' AND table_name = 'store';
```

EXPECTED ERROR (on purpose), step (d)

```
Error Code: 3819.   Check constraint 'chk_zip_digits' is violated.
```

EXPECTED RESULT step (e), 3 rows (order may vary)

CONSTRAINT_NAME	CONSTRAINT_TYPE
PRIMARY	PRIMARY KEY
store_name	UNIQUE
chk_zip_digits	CHECK

☐ **Check:** store_chk_1 is gone; chk_zip_digits rejects '0211A'.

8. Part 4: Temporary Tables and Dropping Tables

Student Guide reference: 3.B.14–3.B.15

To finish the core tasks, you will use a temporary table as a scratch pad to try out a rule without changing the real database. Then you will learn when MySQL will and will not let you drop a table.

☐ Task 4.1 · Test a rule on a scratch table (temporary table) 3.B.14, 3.B.11

Request: *"Marketing wants promo codes with a discount between 1% and 50%. Try the rule out first, but don't add anything permanent to the database yet."*

INSTRUCTIONS

1. Type the `CREATE TEMPORARY TABLE promo_test` statement, including the named `CHECK` constraint.
2. Insert a valid promo code, FALL10 at 10%.
3. **Predict, then run:** insert HALFOFF at 60%. Take a screenshot.
4. Run `SELECT * FROM promo_test;`, then `SHOW TABLES;`. Is `promo_test` listed?

EXPLANATION

Key idea: `CREATE TEMPORARY TABLE` makes a scratch table that MySQL drops automatically at the end of your session.

Why it works: A temporary table accepts the same column definitions and constraints as a regular table, so you can test a rule safely: 60 fails `chk_discount_range` (Error 3819). The table only exists for your current session, so it does not show up in `SHOW TABLES` and never becomes part of the permanent database structure. Your user needs the `CREATE TEMPORARY TABLES` privilege, which your local root account already has.

SQL TO WRITE

```
-- Task 4.1: Test a rule on a scratch table (temporary table)
CREATE TEMPORARY TABLE promo_test
( promo_code   VARCHAR(10) PRIMARY KEY,
  discount_pct TINYINT UNSIGNED NOT NULL,
  CONSTRAINT chk_discount_range CHECK (discount_pct BETWEEN 1 AND 50)
);

INSERT INTO promo_test VALUES ('FALL10', 10);

-- Predict, then run
INSERT INTO promo_test VALUES ('HALFOFF', 60);

SELECT * FROM promo_test;
SHOW TABLES;
```

EXPECTED ERROR (on purpose)

```
Error Code: 3819.   Check constraint 'chk_discount_range' is violated.
```

EXPECTED RESULT 1 row

promo_code	discount_pct
FALL10	10

EXPECTED RESULT SHOW TABLES, 7 rows: no promo_test

Tables_in_beanbrew_dev
category
customer
employee
orders
product
store
store_inventory

☐ **Check:** HALFOFF is rejected; promo_test holds 1 row but is not listed by SHOW TABLES.

☐ Task 4.2 · Drop a table (safely) 3.B.15

Request: *"Clean up your scratch table. And just to be sure: could someone accidentally drop the category table?"*

INSTRUCTIONS

1. (a) **Predict, then run:** `DROP TABLE category;`. Take a screenshot.
2. (b) Drop the scratch table `promo_test`.
3. (c) Try to SELECT from `promo_test` to prove it is gone.

EXPLANATION

Key idea: `DROP TABLE table_name;` deletes the table **and all of its rows**. There is no undo. In Workbench you can also right-click a table and choose Drop Table.

Why it works: `product` still references `category` through `fk_product_category`, so MySQL refuses to drop it (Error 3730). Dependencies protect you. To remove a whole set of related tables you would drop the children first, then the parents (the reverse of Rule 2). `promo_test` has no dependants, so it drops immediately, even before your session ends.

SQL TO WRITE

```
-- Task 4.2: Drop a table (safely)
-- (a) Predict, then run
DROP TABLE category;

-- (b) Drop the scratch table
DROP TABLE promo_test;

-- (c) Prove it is gone
SELECT * FROM promo_test;
```

EXPECTED ERROR (on purpose), step (a)

```
Error Code: 3730.   Cannot drop table 'category' referenced by a foreign key constraint
'fk_product_category' on table 'product'.
```

EXPECTED ERROR, step (c)

```
Error Code: 1146.   Table 'beanbrew_dev.promo_test' doesn't exist
```

☐ **Check:** (a) Error 3730: category is safe. (b) promo_test dropped. (c) Error 1146.

9. Part 5: Challenge Tasks (Optional)

Student Guide reference: 3.B.1–3.B.15

These tasks combine everything in Section 3.B. They are optional but strongly recommended.

☐ Task 5.1 · Link staff to a home store (add a foreign key with ALTER TABLE) 3.B.3, 3.B.5, 3.B.9

Request: *"I want to know which store each employee works at. If a store closes, keep the employee but clear their store."*

INSTRUCTIONS

1. Add a `store_id` column to `employee`. Use the same type as `store.store_id`: `TINYINT UNSIGNED`.
2. Add a named foreign key, `fk_employee_store`, with `ON DELETE SET NULL ON UPDATE CASCADE`.
3. **Predict, then run:** assign Sofia (102) to store 9, which does not exist.
4. Assign Sofia to store 1 (Back Bay), then SELECT all employees.

EXPLANATION

Key idea: `ALTER TABLE t ADD CONSTRAINT name FOREIGN KEY (col) REFERENCES parent(col) ON DELETE … ON UPDATE …;` adds a foreign key to a table that already exists.

Why it works: Foreign keys do not have to be created with the table: `ALTER TABLE` can impose one later (3.B.3). Once it exists, the child column can only hold values that exist in `store`, so store 9 is rejected (Error 1452). `ON DELETE SET NULL` works here because `store_id` allows NULL; it would be invalid on a `NOT NULL` column (3.B.9).

SQL TO WRITE

```
-- Task 5.1: Link staff to a home store
ALTER TABLE employee
ADD store_id TINYINT UNSIGNED;

ALTER TABLE employee
ADD CONSTRAINT fk_employee_store FOREIGN KEY (store_id)
  REFERENCES store(store_id)
  ON DELETE SET NULL ON UPDATE CASCADE;

-- Predict, then run
UPDATE employee SET store_id = 9 WHERE employee_id = 102;

UPDATE employee SET store_id = 1 WHERE employee_id = 102;

SELECT employee_id, first_name, job_title, store_id FROM employee;
```

EXPECTED ERROR (on purpose)

```
Error Code: 1452.  Cannot add or update a child row: a foreign key constraint fails
(`beanbrew_dev`.`employee`, CONSTRAINT `fk_employee_store` FOREIGN KEY (`store_id`)
REFERENCES `store` (`store_id`) ON DELETE SET NULL ON UPDATE CASCADE)
```

EXPECTED RESULT 3 rows

employee_id	first_name	job_title	store_id
100	Dana	Operations Manager	NULL
102	Sofia	Barista	1
103	Ken	Barista	NULL

☐ **Check:** Store 9 is rejected with Error 1452; Sofia's store_id is 1.

☐ Task 5.2 · A sale price below the regular price (two-column CHECK) 3.B.3, 3.B.11–3.B.12

Request: *"We're adding sale prices. A sale price must always be lower than the regular price. Most products won't have one."*

INSTRUCTIONS

1. Add a nullable `sale_price DECIMAL(6,2)` column to `product`.
2. Add a named table-level check, `chk_sale_below_price`, comparing `sale_price` with `unit_price`.
3. Put the Caramel Latte (product 3) on sale at 4.25.
4. **Predict, then run:** set its sale price to 5.95.

5. SELECT products 3 and 4 and look at their sale prices.

EXPLANATION

Key idea: A `CHECK` condition can reference other columns in the same table, for example `CHECK (sale_price < unit_price)`.

Why it works: The rule is tested every time a row is inserted or updated, so 5.95 (above the 4.95 regular price) fails (Error 3819). Products with no sale price pass: comparing NULL gives "unknown", not FALSE, and MySQL only rejects a row when the condition is FALSE. Adding the constraint reports 12 rows affected because every existing row is checked.

SQL TO WRITE

```
-- Task 5.2: A sale price below the regular price
ALTER TABLE product
ADD sale_price DECIMAL(6,2);

ALTER TABLE product
ADD CONSTRAINT chk_sale_below_price CHECK (sale_price < unit_price);

UPDATE product SET sale_price = 4.25 WHERE product_id = 3;

-- Predict, then run
UPDATE product SET sale_price = 5.95 WHERE product_id = 3;

SELECT product_id, product_name, unit_price, sale_price
FROM product
WHERE product_id IN (3, 4);
```

EXPECTED ERROR (on purpose)

```
Error Code: 3819.   Check constraint 'chk_sale_below_price' is violated.
```

EXPECTED RESULT 2 rows

product_id	product_name	unit_price	sale_price
3	Caramel Latte	4.95	4.25
4	Cold Brew	4.50	NULL

☐ **Check:** 4.25 is accepted, 5.95 is rejected; Cold Brew's NULL sale price passes the check.

10. Self-Check

Self-check: row counts after Part 4

Run `SELECT COUNT(*) FROM table;` for each table. Your numbers should match:

Table	Expected rows	Why
category	4	Coffee, Tea, Bakery, Merchandise (category 4 restored in Task 3.5)
product	12	Starter data. The 0.00 product was rejected (3.2); product 11 could not be deleted (3.5)

Table	Expected rows	Why
customer	7	6 starter + Lena Park (2.3); the duplicate Maria was rejected (3.1)
orders	9	Unchanged. No order history was lost!
employee	3	4 starter − Marcus (3.5)
store	2	3 stores − Seaport (3.5); the two bad zip codes were rejected
store_inventory	3	5 rows − 2 cascaded with Seaport (3.5)

Then run `SHOW TABLES;` (7 tables; the temporary promo_test table is gone) and `DESCRIBE` any table you are unsure about.

What to submit

- One script named **Lastname_Firstname_Ex3B.sql** that contains every statement you ran, with a comment above each one (for example `-- Task 3.5 (b)`).
- Stage, Commit, Push to the PracticeExercises repository.